

DOCUMENTO 1: STACK TECNOLÓGICO DEFINITIVO

EQUIPO + STACK

Equipo: 3 Personas

Persona	Rol	Responsabilidad	Stack
P1	Frontend Developer	React PWA, offline, testing E2E	React 18+, TypeScript, Tailwind, Service Workers, Vitest, Playwright
P2	Backend Developer	4 microservicios, DB, integración	FASTapi, Python 3.11, Pytest, Pydantic
P3	Full-Stack/DevOps	AWS, CI/CD, infra, coordinación	Docker Alpine, GitHub Actions, SonarQube, CloudFormation basics

Colaboración: Tareas complejas en conjunto (semanas 1-3, 8-10)

--

STACK COMPLETO

Frontend

Framework: React 18.2+ (TypeScript)
UI: Tailwind CSS v3+ (utility-first)
Build: Vite (no Webpack)
Testing: Vitest (unit) + Playwright (E2E)
State: Zustand (minimal, offline-first)
Storage: IndexedDB (JWT + offline queue)
PWA: Service Workers (offline sync)
Auth: Cognito SDK (@aws-amplify)
Deployment: S3 + CloudFront

Versión Node: 18+ LTS

Backend

Framework: FASTapi 0.104+ (Python 3.11+)

Async: asyncio (built-in)

Validation: Pydantic v2+

ORM: (DynamoDB direct SDK, no ORM)

Testing: Pytest 7.4+ (unit + integration)

Linting: Ruff, Mypy (type checking)

Logger: Structlog (structured logging)

Docker: Alpine Python 3.11 (< 100MB image)

Deployment: Lambda + EC2

Database

Servicio	Uso	Free Tier
DynamoDB	PRIMARY (reportes, users, validations)	25GB ∞
PostgreSQL	EC2 local (Grafana source, logs)	\$0 (en EC2)
S3	Imágenes (pre-signed URLs)	5GB ∞

AWS Services

Servicio	Función	Free Tier
Lambda	3 microservicios (Usuarios, Incidencias, Notificaciones)	1M requests/mes ∞
EC2 t3.micro	1 microservicio FASTapi (Incendios)	750h/mes ∞
API Gateway	Central routing + auth	1M requests/mes ∞
Cognito	Auth + JWT	50k MAU ∞
Rekognition	Validación imágenes (IA)	5k/mes ∞
CloudWatch	Logs + metrics	5GB ∞
SNS	Push notifications	1M/mes ∞
SES	Email alerts	62k/mes ∞
DynamoDB Streams	Event-driven sync	Incluido

DevOps

Herramienta	Propósito	Costo
Docker	Containerization (Alpine Python)	\$0
GitHub	Repo + Actions	\$0
GitHub Actions	CI/CD (test, build, deploy)	\$0
SonarQube Community	Code quality + 60% coverage	\$0
CloudFormation	IaC (basics, no templates complejos)	\$0
CloudWatch	Monitoring	\$0 (5GB logs)

--

MICROSERVICIOS (4 Totales)

Microservicio 1: INCENDIOS (EC2 + FASTapi)

Ubicación: EC2 t3.micro VPC (Subnet privada)

Responsable: P2 (Backend)

Tech: FASTapi + Python

Rutas:

POST /validate → Rekognition API

GET /status → Healthcheck

Entrada: { s3_image_url, report_id }

Salida: { is_fire, confidence, metadata }

DynamoDB: escribir resultado validación

Microservicio 2: USUARIOS (Lambda)

Ubicación: AWS Lambda

Responsable: P2 (Backend)

Tech: Python + Cognito

Rutas:

POST /auth/signup

POST /auth/login

POST /auth/refresh

GET /users/{id}

Handler: Lambda function (1000+ MB RAM)

Auth: Cognito User Pool

DynamoDB: almacenar perfil usuario

Microservicio 3: INCIDENCIAS (Lambda)

Ubicación: AWS Lambda

Responsable: P2 (Backend)

Tech: Python + DynamoDB

Rutas:

POST /reports → crear reporte

GET /reports → listar reportes

Handler: Lambda function

DynamoDB: escribir reporte + dispara Streams

Output: event a Notificaciones

Microservicio 4: NOTIFICACIONES (Lambda)

Ubicación: AWS Lambda (async, Streams trigger)

Responsable: P2 (Backend) + P3 (SNS/SES config)

Tech: Python + SNS + SES

Trigger: DynamoDB Streams (cuando reporte se crea)

Funciones:

- Calcular radio 5km
- Query usuarios en zona
- SNS push notification
- SES email alert

DynamoDB: leer reportes

SNS: publicar

SES: enviar email

API Gateway (Central)

Responsable: P3 (DevOps)

Tech: AWS API Gateway

Routes:

/auth/* → Lambda Usuarios
/reports/* → Lambda Incidencias
/validate/* → EC2 Incendios
/notify/* → Lambda Notificaciones

Auth: Cognito authorizer (JWT)
Rate limit: 1000 requests/min
CORS: enabled para S3 CloudFront

--

SEGURIDAD (Sin IAM, solo Security Groups)

EC2 Security Group

Inbound:

- Port 443 (HTTPS) from API Gateway
- Port 22 (SSH) from 0.0.0.0 (dev only)

Outbound:

- Port 443 (HTTPS) → Rekognition, S3, DynamoDB
- Port 5432 (PostgreSQL) → localhost only

VPC: Privada (subnet)

NAT Gateway: para outbound internet

S3 Bucket

Bucket: privado (no public read)
Objects: Pre-signed URLs (15 min TTL)
Lifecycle: Delete imágenes después de 24 horas
CORS: enabled para CloudFront

Authentication Flow

Frontend → Cognito (JWT)
Lambda: valida JWT vía Cognito
EC2: recibe JWT, valida firma local (caché)
Offline: JWT en IndexedDB + sync en reconexión

--



VERSIONES ESPECÍFICAS

Tecnología	Versión	Razón
Python	3.11+	LTS, mejor performance
React	18.2+	Latest stable
FASTapi	0.104+	Async support
Node (for build)	18+ LTS	Docker Alpine support
Pytest	7.4+	Async fixture support
Docker	Latest	Alpine base
AWS SDK Python	1.28+	Latest features

--

--



STACK VALIDADO

- ✓ Coherencia: Todo Python backend (FASTapi + Lambda + Dockerfile)
- ✓ Escalabilidad: Serverless + 1 EC2 (híbrida)
- ✓ Costo: \$0 infraestructura (free tier permanente)
- ✓ Educativo: Aprenden real AWS ecosystem
- ✓ Profesional: Patrones reales (offline, streams, security)
- ✓ Tiempo: 10 semanas realizables (3 personas)